

第2章: TypeScript入門

アプリの「動き（ロジック）」はすべて **TypeScript** という言語で書きます。この章では、後の章で出てくるコードを読み書きできるようになるための、TypeScriptの基礎を完全初心者向けに解説します。**初めて出てくる文法はすべて1つずつ説明**します。

1. TypeScript とは何か

1.1 JavaScript と TypeScript の関係

世の中のWebやアプリの多くは **JavaScript**（ジャバスクリプト） という言語で動いています。**TypeScript**（タイプスクリプト） は、その JavaScript に「**型**（かた、**type**）」という仕組みを足した、上位互換の言語です。

- **JavaScript** ... 古くからある、型のない言語。手軽だがミスに気づきにくい。
- **TypeScript** ... JavaScriptに型を足した言語。Microsoft社が開発。書いた瞬間にミスを発見できる。

「**上位互換**」とは？ TypeScriptはJavaScriptの全機能を含みつつ、さらに型を足したもの、という意味です。だからJavaScriptが書ければTypeScriptも書けますし、最終的にTypeScriptは「型を取り除いてJavaScriptに変換（コンパイル）」されてから実行されます。

1.2 「型」とは何か — みかん箱のたとえ

型（type） とは「この箱（変数）には、この種類のデータしか入れてはいけない」というルールです。

身近な例え:「みかん専用」と書かれた箱に、りんごを入れようすると「それはみかんじゃないよ！」と止めてくれる。これが型の働きです。プログラムでも、文字を入れるべき場所に数値を入れようすると、**実行する前に**エラーで教えてくれます。

```
// =====
// 「型あり」と「型なし」で何が違うかの比較
// =====
// 注: // で始まる行は「コメント」。プログラムとしては実行されず、説明用を書ける。

// ----- 型なし (JavaScript) の場合 -----
function add(a, b) {           // function = 関数を作るキーワード / add = 関数名 / (a, b) = 受け取る材料2つ
    return a + b;              // return = 結果を返す / a + b = 足し算 (だが...)
}
add("1", 2);                   // "1"は文字列、2は数値。JSは文字列連結して "12" を返してしまう (バグ!)
```

```
// ----- 型あり (TypeScript) の場合 -----
function addTs(a: number, b: number): number { // a: number → aは数値型のみ / : number (最後) → 戻り値も数値
    return a + b;                  // a も b も数値なので、必ず正しい足し算になる
}
addTs("1", 2);                    // ← "1"は文字列なので、ここで「型エラー」が表示され、実行前にミスに気づける
// VS Codeでは addTs("1", 2) の "1" の下に赤い波線が引かれ、次のメッセージが出る:
// Argument of type 'string' is not assignable to parameter of type 'number'.
// ('string' 型の引数は 'number' 型のパラメータに代入できません)
```

「コンパイル」とは？ 人間が書いたTypeScriptを、コンピュータが実行できるJavaScriptに**変換する処理**のこと。この変換のときに型チェックが行われ、間違いがあればエラーになります。「実行前のチェック」だと考えてください。

2. 変数 — 値に名前を付けて入れておく箱

2.1 `const` と `let`

変数（へんすう、variable）とは「値に名前を付けて保存しておく箱」です。TypeScriptでは主に2つのキーワードで変数を作ります。

```
const title = "リーダブルコード";
// const : 「後から中身を変えない箱」を作るキーワード (constant=定数 の略)
// title : 変数名 (箱の名前)。自分で好きに付けられる。半角英数字で、小文字始まりが慣習
// =      : 「右の値を左の箱に入れる」という代入の記号 (イコールだが"等しい"ではない)
// "リーダブルコード" : 入れる値。ダブルクォートで囲むと「文字列」になる
// ;      : 文の終わりを示すセミコロン
```

```
let count = 0;           // let : 「後から中身を変えられる箱」を作るキーワード / 0 は数値
count = 1;               // 中身を 0 から 1 に変更できる (letだから可能)
// もし上が const count = 0; だったら、count = 1; の行でエラーになる (constは変更不可のため)
```

const と let の使い分け: 迷ったらまず **const** を使うのが鉄則です。「変更しない」とコードで宣言しておくことで、うっかり書き換えるミスを防げます。本当に後で変える必要がある場合だけ **let** にします。(昔の **var** というキーワードもありますが、現在は使いません。)

3. 基本の型

TypeScriptでよく使う基本的な型を紹介します。変数名の後ろに **: 型名** と書くと、その変数の型を明示できます。

```
const bookTitle: string = "達人プログラマー"; // string (ストリング) = 文字列の型
const publishYear: number = 1999;              // number (ナンバー) = 数値の型 (整数・小数ど
ちらも)
const isRead: boolean = true;                  // boolean (ブーリアン) = 真偽値の型。true
(真) か false (偽) の2択
// : string や : number の部分が「型注釈 (type annotation)」。「この箱の中身の種類」を宣言して
いる
```

型	意味	例
<code>string</code>	文字列 (文字の並び)	<code>"こんにちは"</code> , <code>"鈴木"</code>
<code>number</code>	数値 (整数・小数)	<code>42</code> , <code>3.14</code> , <code>2024</code>
<code>boolean</code>	真偽値 (はい/いいえ)	<code>true</code> , <code>false</code>
<code>null</code>	「値が無い」ことを表す特別な値	<code>null</code>
<code>undefined</code>	「まだ値が設定されていない」状態	<code>undefined</code>

型注釈は省略できる: 実は `const title = "本"` と書くだけで、TypeScriptは「右が文字列だから title は string 型だ」と自動で推測してくれます (これを「型推論」と呼びます)。なので毎回 `: string` と書く必要はありません。本書では分かりやすさのため、要所で型注釈を明示します。

4. 配列とオブジェクト — 複数のデータをまとめる

4.1 配列（はいれつ、array） — 値の並び

配列 は「同じ種類の値を順番に並べたリスト」です。[]（角カッコ）で囲みます。

```
const titles: string[] = ["本A", "本B", "本C"];
// string[] : 「文字列(string)の配列([])」という型。「文字列がいくつか並んだリスト」の意味
// [ ... ] : 配列を作る記号。中身はカンマ , で区切る
// 並んだ各値を「要素 (element)」と呼ぶ

console.log(titles[0]); // titles[0] → 配列の「0番目」の要素を取り出す。結果: "本A"
// 注意: プログラミングの数は0から始まる! 1番目=[0]、2番目=[1]、3番目=[2]
// console.log(...) : ( )の中身をターミナルやデバッグ画面に表示する命令。動作確認によく使う
```

4.2 オブジェクト（object） — 「キーと値」の組

オブジェクト は「名前（キー）と値の組」をまとめたものです。{ }（中カッコ）で囲みます。1冊の本の情報をまとめるのに最適です。

```
const book = {
  title: "リーダブルコード", // title というキー（項目名）に "リーダブルコード" という値
  author: "Dustin Boswell", // author というキーに著者名。各組はカンマ , で区切る
  year: 2012, // year というキーに数値
  isRead: true, // isRead というキーに真偽値
};
// { } : オブジェクトを作る記号 / キー: 値 の形で項目を並べる

console.log(book.title); // book.title → オブジェクトの title の値を取り出す。結果: "リーダブルコード"
// . (ドット) : 「オブジェクトの中の、その名前の項目」にアクセスする記号
```

5. 型に名前を付ける — type と interface

同じ形のオブジェクト（例: 本の情報）を何度も扱うとき、その「形」に名前を付けておくと便利で安全です。本書では主に type を使います。

```
// type : 型に名前を付けるキーワード / Book : 付ける名前 (型名は大文字始まりが慣習)
type Book = {
  id: string;      // id (識別番号) は文字列
  title: string;   // title (タイトル) は文字列
  author: string;  // author (著者) は文字列
  status: string;  // status (読書状態) は文字列
  isRead: boolean; // isRead (読了したか) は真偽値
};
// ↑ これで「Book型 = id, title, author, status, isReadを持つオブジェクト」と定義できた

// 定義したBook型を使って変数を作る
const myBook: Book = {           // : Book → 「この変数はBook型ですよ」と宣言
  id: "1",
  title: "達人プログラマー",
  author: "David Thomas",
  status: "読書中",
  isRead: false,
};
// もしここで title を書き忘れると、「title が足りません」とエラーになる (型のおかげ)
```

type と interface の違い: 同じ型に名前を付ける **interface** (インターフェース) というキーワードもあります。オブジェクトの形を定義する用途ではほぼ同じことができます。本書では一貫して **type** を使いますが、他の教材で **interface Book { ... }** を見ても「同じようなもの」と理解してください。

5.1 「省略可能」を表す ?

「あってもなくてもよい項目」は、キー名の後ろに ? (クエスチョンマーク) を付けます。

```
type Book = {
  id: string;
  title: string;
  memo?: string; // memo? → memoは「あってもなくてもよい」項目 (省略可能、optional)
};

const book1: Book = { id: "1", title: "本A", memo: "面白い" }; // memoあり: OK
const book2: Book = { id: "2", title: "本B" };                 // memoなし: これもOK (?のおかげ)
```

6. 関数 — 処理に名前を付けてまとめる

関数 (かんすう、function) は「一連の処理に名前を付けてまとめたもの」です。呼び出すと中の処理が動きます。

6.1 基本の書き方

```
// function : 関数を作るキーワード
// greet    : 関数名 / (name: string) : nameという文字列の引数を1つ受け取る / : string : 戻り値も文字列
function greet(name: string): string {
  return "こんにちは、" + name + "さん"; // return : 結果を返す / + : 文字列をつなぐ(連結)
}

const message = greet("鈴木"); // greet関数を呼び出し、"鈴木"を渡す。戻り値が message に入る
console.log(message);          // 結果: こんにちは、鈴木さん
```

「引数（ひきすう、argument）」と「戻り値（return value）」:- 引数: 関数に渡す材料。greet("鈴木") の "鈴木" の部分。- 戻り値: 関数が処理結果として返す値。return の後ろに書いた値。

6.2 アロー関数 — もう一つの書き方

React/React Nativeでは、=>（矢印のような記号）を使ったアロー関数という短い書き方を非常によく使います。覚えておきましょう。

```
// const greet : 関数を変数に入れる形 / (name: string): string : 引数と戻り値の型 / => : アロー関数（矢印）
const greet = (name: string): string => {
  return "こんにちは、" + name + "さん";
};
// 上の function版とまったく同じ動作。React Nativeではこのアロー関数の形が主流
```

=> の読み方: 「アロー（arrow = 矢印）」と読みます。（引数）=> { 処理 } で「引数を受け取って処理する関数」を表します。一見とっつきにくいですが、慣れると簡潔で読みやすい書き方です。

7. よく使う配列の操作 — map と filter

アプリでは「本のリストを画面に並べる」「読了した本だけ抜き出す」といった操作を頻繁に行います。これに使うのが map と filter です。第7章以降で必ず出てくるので、ここで基礎を押さえます。

7.1 **map** — 各要素を変換して新しい配列を作る

```
const numbers = [1, 2, 3]; // 数値の配列

const doubled = numbers.map((n) => n * 2); // map : 各要素に処理をして新しい配列を作る
// .map(...) : 配列の各要素に対して、( )内の関数を順番に適用する
// (n) => n * 2 : 「受け取った n を 2倍して返す」アロー関数。nには1,2,3が順に入る
// 結果: doubled は [2, 4, 6] になる (元のnumbersは変わらない)
```

React Nativeでの **map** の使いどころ: 「本の配列」を「画面に表示する本カードの配列」に変換するのにこの **map** を使います。第7章で実際に登場します。

7.2 **filter** — 条件に合う要素だけ残す

```
const books = [
  { title: "本A", isRead: true },
  { title: "本B", isRead: false },
  { title: "本C", isRead: true },
];

const readBooks = books.filter((b) => b.isRead === true); // filter : 条件に合う要素だけ
抜き出す
// .filter(...) : 各要素のうち、( )内の関数が true を返したのだけ残す
// (b) => b.isRead === true : 「その本の isRead が true か?」を判定する関数
// === : 「左右が厳密に等しいか」を判定する比較演算子 (= が1つだと代入になるので注意)
// 結果: readBooks は 本A と 本C の2つだけの配列になる
```

=== (イコール3つ) に注意: = (1つ) は「代入 (箱に入れる)」、**=== (3つ)** は「等しいか判定する」です。初心者がよく間違えるポイントなので意識しましょう。「等しいかを聞きたいときは=を3つ」と覚えてください。

8. 条件分岐 `if` と 三項演算子

8.1 `if` 文 — 条件によって処理を分ける

```
const status = "読了";

if (status === "読了") {           // if : 「もし(条件)なら」 / ( )内が条件 / === で等しいか判定
  console.log("読み終わった本です"); // 条件が true のとき、この { } 内が実行される
} else {                           // else : 「そうでなければ」
  console.log("まだ読んでいません"); // 条件が false のとき、こちらが実行される
}

// 結果: status が "読了" なので「読み終わった本です」が表示される
```

8.2 三項演算子 — 短い条件分岐

React Nativeの画面コードでは、`if` 文の代わりに三項演算子という1行で書ける条件分岐をよく使います。

```
const isRead = true;
const label = isRead ? "読了" : "未読";
// 条件 ? Aの値 : Bの値 という形
// isRead (条件) が true なら "読了"、false なら "未読" が label に入る
// ? の前が条件、? と : の間が「真のときの値」、: の後が「偽のときの値」
// 結果: label は "読了"
```

なぜ三項演算子を使う？ React Nativeの画面コードの中（後の章で出てくるJSX）では、`if` 文がそのままでは書きにくい場面があります。そこで「条件 ? A : B」の三項演算子が画面の表示切り替えに重宝します。第4章以降で実際に使います。

9. ジェネリクス — 型に「中身の型」を渡すしくみ（`<...>`）

後の章では `useState<Book[]>(...)` や `Promise<Book[]>`、`useLocalSearchParams<{ id: string }>`（`()`）のように、型名のうしろに山カッコ `<...>` が付く書き方が頻繁に出てきます。これをジェネリクス（generics）と呼びます。初出でいきなり出るとギョツとするので、ここで意味を押さえておきましょう。

ジェネリクスは「入れ物の型に、「中に入る物の型」を渡す」しくみです。身近なたとえで言うと、「箱」という型に「何を入れる箱か（みかん用／本用）」を指定するイメージです。


```
// 配列の型 string[] は、実は Array<string> とも書ける (同じ意味)
// Array      : 「配列」という"入れ物"の型
// <string>    : 「その配列の中身は文字列(string)ですよ」と中身の型を渡している
const titles: Array<string> = ["本A", "本B"]; // string[] と全く同じ意味

// 中身を Book にすれば「本の配列」になる
// (Book型は第6章で定義します。「本1冊分の形」と思ってください)
// const books: Array<Book> = [...]; // Book[] と同じ
```

- `<...>` の中に書いた型が「中身の型」になります。 `Array<string>` は「文字列の配列」、 `Array<Book>` は「本の配列」です。
- 同じ「配列」という入れ物でも、 `<string>` か `<Book>` かで「中に何が入るか」が変わります。これがジェネリクス（＝汎用的に、中身の型を後から指定できる）の便利さです。

後の章での登場例（今は眺めるだけでOK）:- `useState<Book[]>()` ... 「Bookの配列を持つstate」を作る（第7章）。 `<Book[]>` で中身の型を指定。- `Promise<Book[]>` ... 「最終的にBookの配列を返す非同期処理」の型（下の第9.5節・第6章）。- `useLocalSearchParams<{ id: string }>()` ... 「id という文字列を受け取る」と中身の型を指定（第8章）。

いずれも「`<...>`」の中は「中身の型」を教えているんだな」と分かれれば読めます。自分でジェネリクスを設計する必要は当面ありません。「既にある入れ物に中身の型を渡すだけ」と考えてください。

9.5 非同期処理 — `async` / `await` と `Promise`

アプリは「サーバーからデータを取ってくる」という時間のかかる処理を行います。データが届くまで他の処理を止めずに待つ仕組みが `async`（エイシク） / `await`（アウェイト）です。第6・7章でSupabaseからデータを取得するときに必ず使うので、概念をしっかり掴んでおきましょう。

```
// async : 「この関数の中には"待つ処理"が含まれる」という目印を付けるキーワード
async function loadBooks() {
  // await : 「この処理が終わるまで、ここで待つ」という指示
  // fetchBooksFromServer() はサーバーからデータを取る（時間がかかる）関数だと考えてください
  const books = await fetchBooksFromServer(); // データが届くまで待ってから、結果を books
  // に入れる
  console.log(books); // 届いたデータを表示
}
```

- ① `await` を付けないとどうなる？ `await` は「結果が届くまでこの行で待つ」という指示です。もし `await` を付け忘れると、プログラムは「データが届くのを待たずに次の行へ進んで」しまいます。すると `books` にはまだデータが入っておらず、空

っぱや「処理中の状態」を表示してしまう、というバグになります。「時間のかかる処理の前には `await` 」とセットで覚えてください。なお `await` は `async` を付けた関数の中でしか使えません（だから2つはセットで登場します）。

② **Promise**（プロミス）とは？「時間のかかる処理」は、呼んだ瞬間には結果がまだありません。その「結果は後で渡すよ、という"引換券"」のことを **Promise** と呼びます。レストランで料理を注文したときの「番号札」のイメージです。

Promiseの3つの状態（やさしく）：Promiseは次のどれかの状態を持ちます。- **待機中（pending）** ... まだ料理ができていない（処理の途中）。- **成功（fulfilled）** ... 料理ができて受け取れた（データが届いた）。- **失敗（rejected）** ... 料理が作れなかった（通信エラーなど）。

`await` は「この番号札（Promise）が"成功"か"失敗"になるまで待って、中身（料理＝データ）を受け取る」働きをします。失敗したときの受け止め方が、第7章で学ぶ `try / catch` です。

③ **Promise<Book[]>** の読み方（ジェネリクス再登場） 第6章では `function fetchBooks()`：
Promise<Book[]> のような書き方が出ます。これは第9節のジェネリクスです。「**Promise<Book[]>**」は「最終的に **Book** の配列（**Book[]**）を渡してくれる引換券」という意味。`async` を付けた関数の戻り値は、必ずこの **Promise<...>** の形になります（中身が無いなら **Promise<void>**。`void` は「返す値なし」の型）。

身近な例え（再）：カップ麺にお湯を入れて「3分待つ」のが `await`、その「3分後にできあがる」という約束が **Promise** です。`async` は「この調理には待ち時間がありますよ」という札。待っている間に他のこともできるのが非同期（asynchronous＝同時並行的）の良さです。今は「サーバー通信には `async/await` を使い、戻り値は Promise になる」と分かればOKです。

10. この章のまとめ

この章では、後の章のコードを読むために必要なTypeScriptの基礎を学びました。

- 変数: `const`（変更しない） / `let`（変更する）。迷ったら `const`
- 基本の型: `string`（文字列） / `number`（数値） / `boolean`（真偽値）
- 配列 `[]` と オブジェクト `{ }` でデータをまとめる
- `type` で「本の形（Book型）」のような型に名前を付ける。`?` で省略可能
- 関数 と アロー関数 `=>`。React Nativeではアロー関数が主流
- `map`（変換） / `filter`（抽出）は一覧表示で多用する
- `if` と 三項演算子 `条件 ? A : B` で条件分岐
- ジェネリクス `<...>`（例: `useState<Book[]>`）は「入れ物の型に"中身の型"を渡す」しくみ
- `async` / `await` でサーバー通信などの「待つ処理」を書く。戻り値は **Promise<...>**

全部覚えなくて大丈夫: ここで紹介した文法は、後の章で実際のコードに何度も出てきます。そのたびにこの第2章に戻って確認すれば、自然と身につきます。次の第3章では、これらを使って画面の部品を作る **React** を学びます。